PROJECT REPORT | BACKEND DEVELOPMENT

Course API

REST API with Express.js, MongoDB and JWT Authentication

Node.js | Express 4 | MongoDB | Mongoose 8 | JWT | Postman

STUDENT	**Muhammad Furqan**
ROLL NO.	**SU92-BSCSM-F24-020**
SECTION	**5-A**
SUBJECT	**Advance Computer Programming**

Report date: 26 October 2026

Includes Postman test evidence

SECTION 01

What is this project?

Course API is a backend web service: a **REST API** that other apps (a website, a mobile app, Postman) talk to over HTTP. It lets users **register and log in**, and lets logged-in users manage **course categories** and **courses**. Data is stored in **MongoDB**. The code follows a clean **modular MVC** layout (models, controllers, routes, middleware).

The Postman PDF you uploaded is the **testing evidence** for this API: screenshots proving the endpoints return the correct status codes (201, 200, 400, 404). This report documents the whole project and that evidence in one place.

13	3	3	6
API ENDPOINTS	DATA MODELS	MIDDLEWARE	POSTMAN PROOFS

What it can do

- **Authentication:** sign up, log in, and fetch the current user with a signed JWT token.
- **Categories:** create, list, view, update and delete course categories.
- **Courses:** full CRUD on courses, each linked to a category, with a `?category=` filter.
- **Safe by design:** hashed passwords, protected routes, validation messages and one central error handler.

Tech stack

Layer	Technology	Purpose
Runtime	Node.js	JavaScript on the server
Framework	Express 4.19	Routing, JSON body parsing, middleware
Database	MongoDB + Mongoose 8.5	Stores users, categories and courses; schema validation
Auth	jsonwebtoken 9, bcryptjs 2.4	Signed tokens and password hashing
Config	dotenv, cors	Environment variables, cross-origin access
Dev tools	nodemon, Postman	Auto-restart in dev, manual API testing

SECTION 02

Architecture and structure

The project separates responsibilities so each file does one job. A request flows through the layers below.

1 Client	2 Route	3 Middleware	4 Controller	5 Model
Postman / app	matches URL	JWT check	business logic	MongoDB

Any error thrown along the way is caught by **asyncHandler** and turned into clean JSON by the **errorHandler**.

Folder layout

```
course-api/
|-- server.js                starts DB connection, then the server
|-- package.json  .env.example  README.md
`-- src/
    |-- app.js               Express setup, routes, 404 handler
    |-- config/
    |   `-- db.js            MongoDB connection
    |-- models/              User, Category, Course
    |-- controllers/         authController, categoryController, courseController
    |-- routes/              authRoutes, categoryRoutes, courseRoutes
    `-- middleware/          auth (JWT), asyncHandler, errorHandler
```

Data models

Model	Field	Type	Rules
User	name	String	required, trimmed
	email	String	required, unique, lowercase, valid format
	password	String	required, min 6 chars, hidden by default, hashed with bcrypt (10 rounds)
Category	name	String	required, unique, trimmed
	description	String	optional, default empty
Course	title	String	required, trimmed
	description / price	String / Number	optional; price cannot be negative, default 0
	category	ObjectId	required, references Category

All three models use `timestamps: true`, so `createdAt` and `updatedAt` are added automatically.

SECTION 03

Endpoint reference

Protected routes need the header `Authorization: Bearer <token>`.

Method	URL	Access	What it does	Status codes
POST	/api/auth/register	Public	Create account, returns token	201, 400
POST	/api/auth/login	Public	Log in, returns token	200, 400, 401
GET	/api/auth/me	Required	Current user profile	200, 401
POST	/api/categories	Required	Create category	201, 400, 401
GET	/api/categories	Public	List categories (newest first)	200
GET	/api/categories/:id	Public	Get one category	200, 404
PUT	/api/categories/:id	Required	Update category	200, 400, 401, 404
DELETE	/api/categories/:id	Required	Delete (blocked if courses use it)	200, 400, 401, 404
POST	/api/courses	Required	Create course in a category	201, 400, 401, 404
GET	/api/courses	Public	List courses, optional ?category=	200
GET	/api/courses/:id	Public	Get one course	200, 404
PUT	/api/courses/:id	Required	Update course	200, 400, 401, 404
DELETE	/api/courses/:id	Required	Delete course	200, 401, 404

Example request bodies

```
POST /api/auth/register
  { "name": "Muhammad Furqan", "email": "furqan@example.com", "password": "123456" }
POST /api/auth/login
  { "email": "furqan@example.com", "password": "123456" }
POST /api/categories
  { "name": "Web Development", "description": "Frontend and backend courses" }
POST /api/courses
  { "title": "Node.js Basics", "price": 49, "category": "<categoryId>" }
```

SECTION 04

Validation, security and error handling

Every failure returns the same JSON shape: `{ "success": false, "message": "..." }`, so clients can handle errors in one place.

Situation	Status	Message / behaviour	Where
Missing required field	400	"name is required", "email, password is required"...	Controllers
Invalid JSON body	400	"Invalid JSON body"	errorHandler
Mongoose validation failed	400	Joined validation messages + errors array	errorHandler
Duplicate unique value	400	"email already exists" (code 11000)	errorHandler
Email already registered	400	"Email already registered"	authController
Wrong email or password	401	"Invalid email or password"	authController
Token missing / invalid / expired	401	"Not authorized, ..."	auth middleware
Invalid ID format (CastError)	404	"Resource not found (invalid ID)"	errorHandler
Record not found	404	"Category not found" / "Course not found"	Controllers
Unknown route	404	"Route not found: METHOD /url"	app.js

Security features

- **Password hashing:** bcrypt (10 rounds) in a pre-save hook; passwords are never returned (`select: false`).
- **JWT auth:** tokens signed with `JWT_SECRET`, 7-day expiry by default; the middleware also confirms the user still exists.
- **Secrets stay out of code:** configuration lives in `.env` (only `.env.example` is shared).
- **Safe deletes:** a category cannot be deleted while courses still use it.
- **Referential checks:** creating or updating a course verifies the category ID is valid and exists.
- **Async safety:** `asyncHandler` forwards any rejected promise to the central error handler instead of crashing.

SECTION 05

Everything completed

01 Project setup — DONE

- Initialised Node project with scripts `start` and `dev` (nodemon)
- Installed express, mongoose, jsonwebtoken, bcryptjs, cors, dotenv
- Added `.env.example`, `.gitignore` and a README with setup steps

02 Core application — DONE

- Built `server.js` (connect DB, then listen on PORT, default 5000)
- Built `app.js` with CORS, JSON parsing, health route, route mounting and 404 handler
- Wrote `config/db.js` with connection logging and exit on failure

03 Data layer — DONE

- Created User, Category and Course Mongoose schemas with validation and timestamps
- Password hashing hook and `comparePassword` method on User

04 Authentication — DONE

- Register, login and get-me endpoints with token signing
- `protect` middleware verifying Bearer tokens

05 Resources — DONE

- Categories: create, list, get, update, delete (with in-use protection)
- Courses: create, list (with category filter), get, update, delete, populated category data

06 Quality — DONE

- Central error handler and async wrapper
- Consistent JSON responses and correct HTTP status codes across the API

07 Testing and evidence — DONE

- Tested registration, login, category create/list in Postman
- Tested validation failure (400) and unknown route (404)
- Compiled the six screenshots into a labelled evidence PDF (name, roll no., section, subject)

SECTION 06

Postman test results

Six requests were run against `http://localhost:5000`. All returned the status the API is designed to return, including the two intentional failure cases.

#	Test	Request	Expected	Actual	Result
1a	**Register user**	`POST /api/auth/register`	201	**201 Created**	**PASS**
1b	**Login**	`POST /api/auth/login`	200	**200 OK**	**PASS**
2	**Create category**	`POST /api/categories`	201	**201 Created**	**PASS**
3	**List categories**	`GET /api/categories`	200	**200 OK**	**PASS**
4	**Validation (no name)**	`POST /api/categories {}`	400	**400 Bad Request**	**PASS**
5	**Unknown route**	`GET /api/this-route-does-not-exist`	404	**404 Not Found**	**PASS**

Reading the results

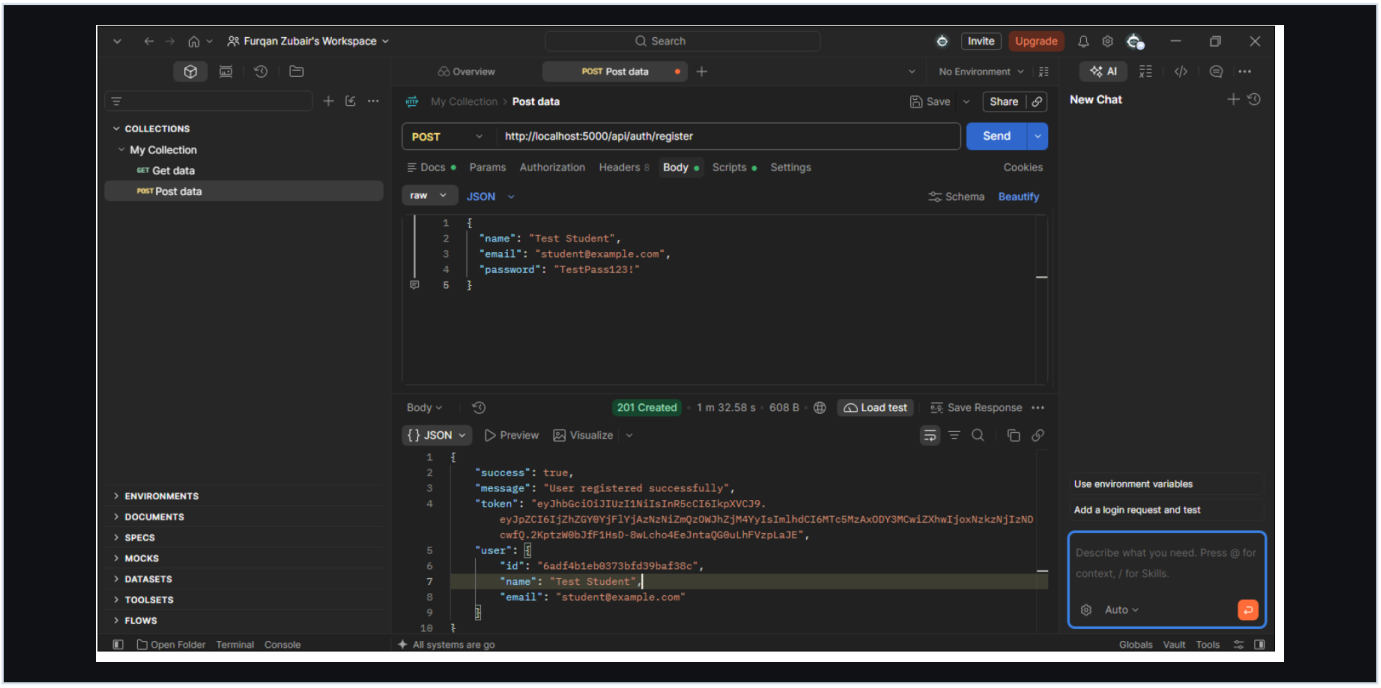
Tests 4 and 5 are **meant** to fail. A 400 for a missing name and a 404 for an unknown route prove the validation and error handling work. Response times were 73 to 374 ms; the first register call took about 1 min 32 s (most likely first-request warm-up).

SECTION 07

Evidence screenshots

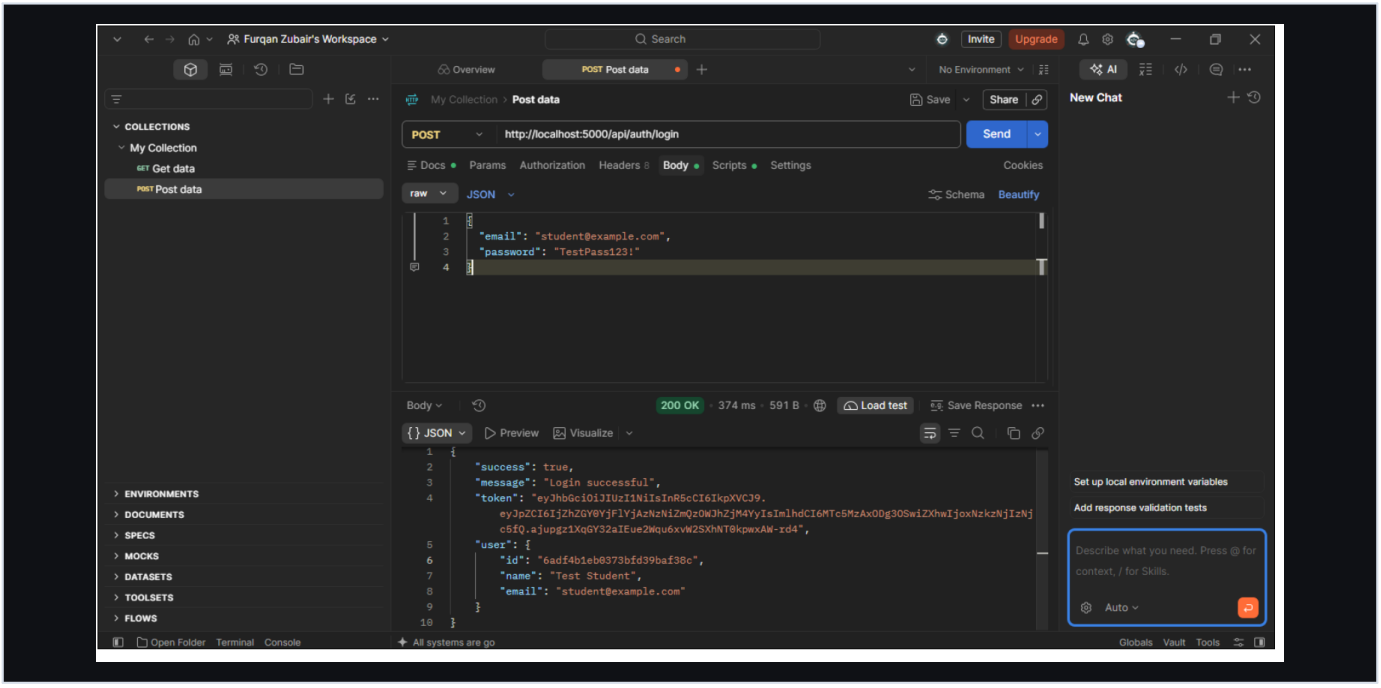
Screenshot 1 (a): Register - 201 Created

Sends name, email and password. The API creates the user, hashes the password and returns a success message, a JWT token and the user object (id, name, email).



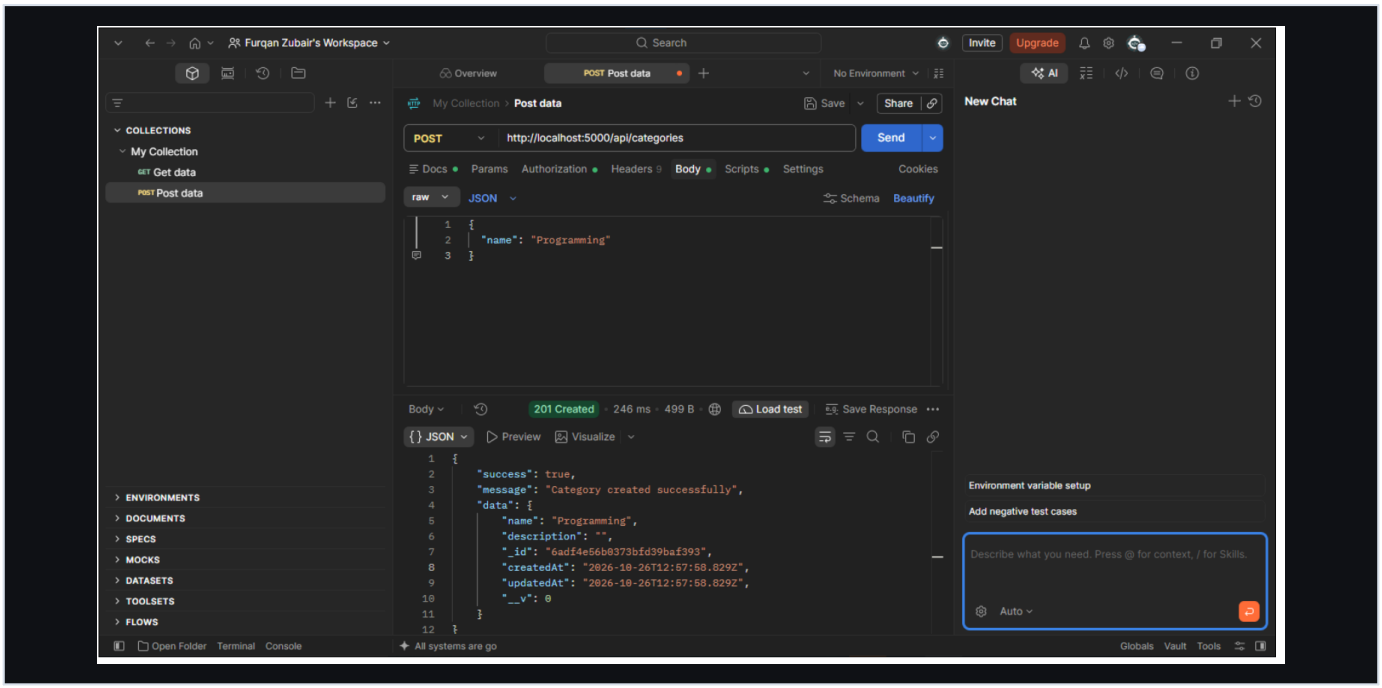
Screenshot 1 (b): Login - 200 OK

Logs in with the same credentials and receives a fresh token in 374 ms. The password is never included in the response.



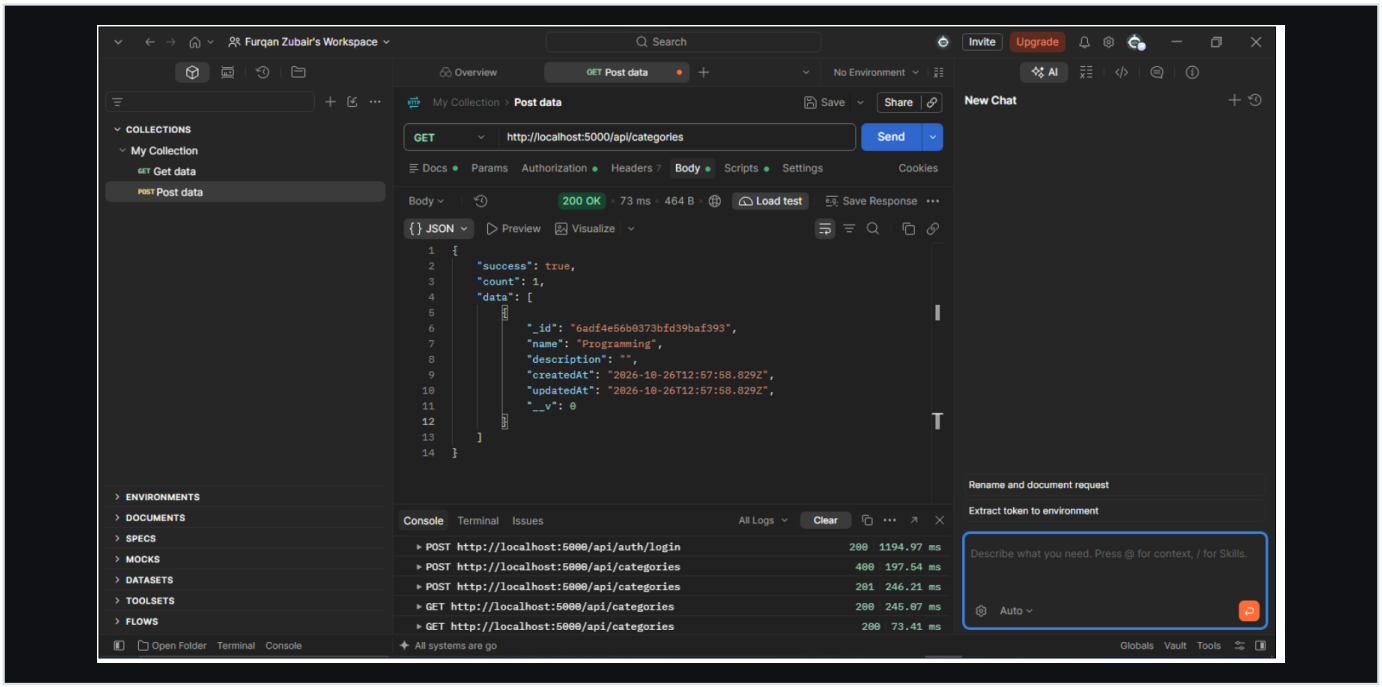
Screenshot 2: Create category - 201 Created

Creates the category "Programming". The response shows the saved document with _id, timestamps and the default empty description.



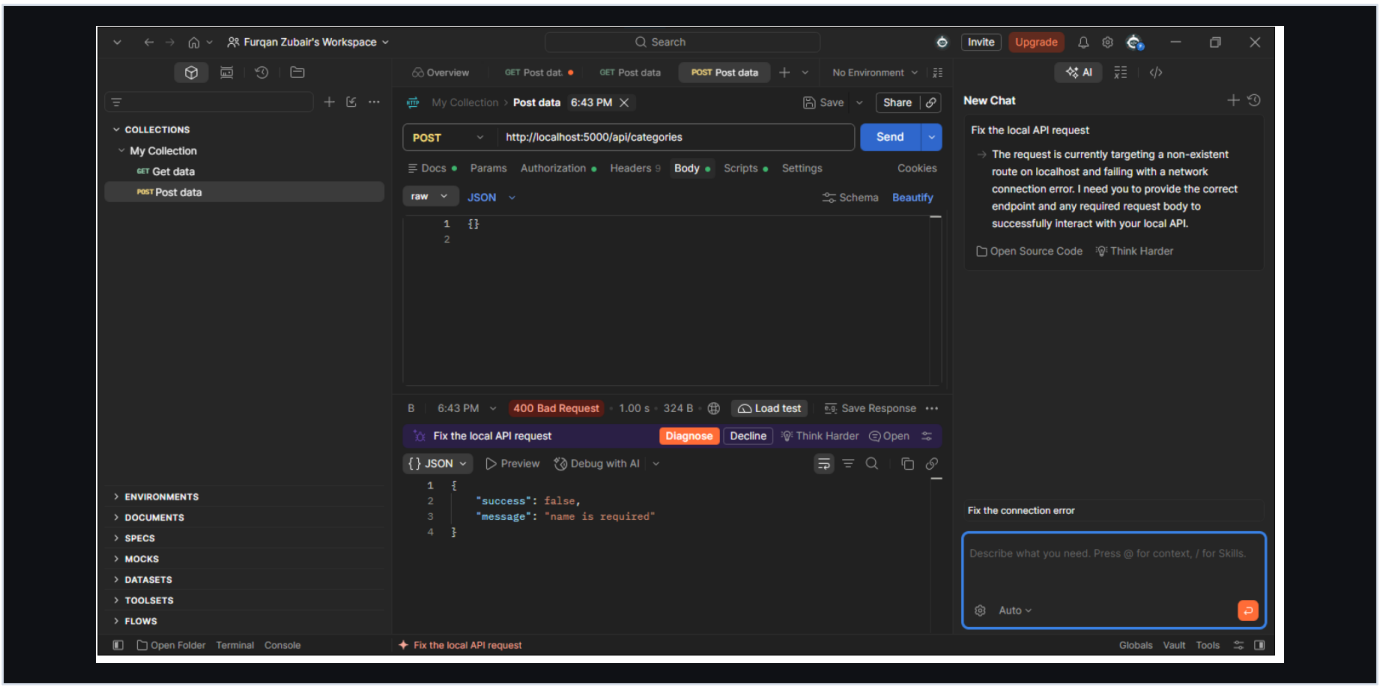
Screenshot 3: List categories - 200 OK

Returns count 1 and the new category in the data array. The console panel also shows the full request history of the session.



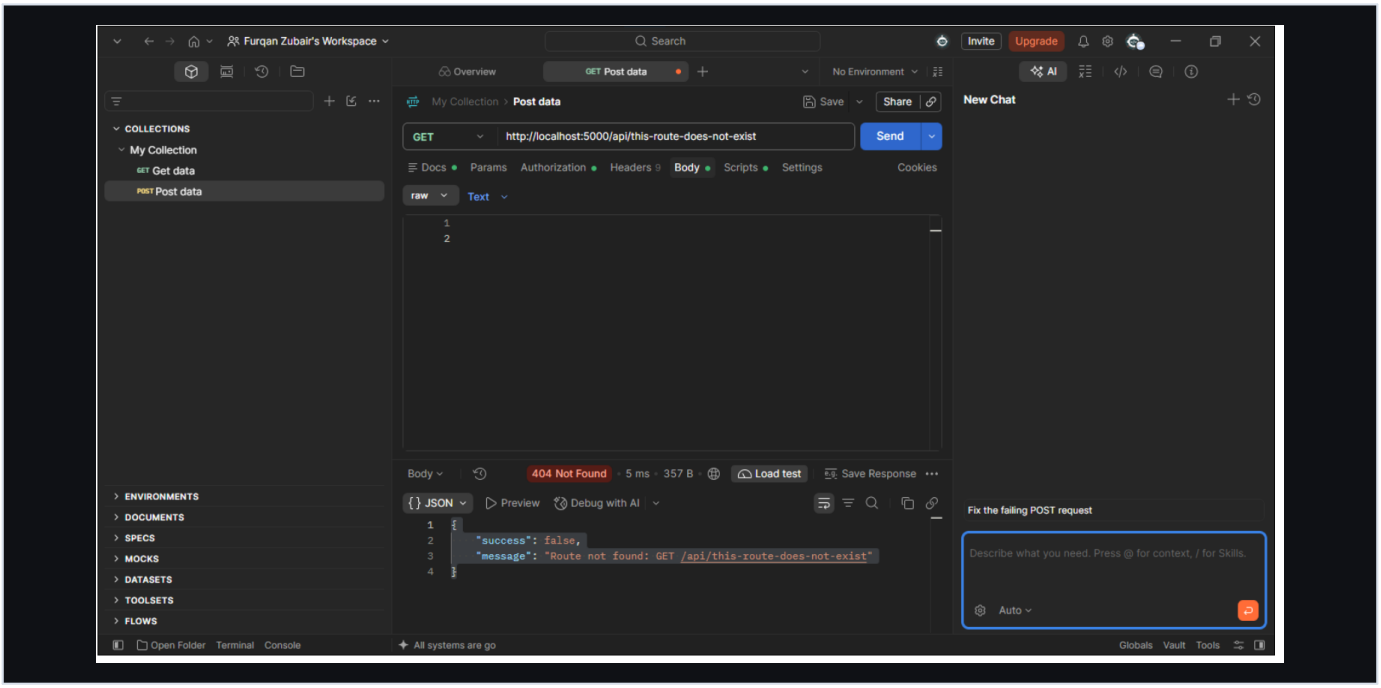
Screenshot 4: Missing name - 400 Bad Request

Posts an empty body {}. The API rejects it with "name is required", proving the controller validation works.



Screenshot 5: Unknown route - 404 Not Found

Requests a route that does not exist. The catch-all handler answers with "Route not found: GET /api/this-route-does-not-exist".



SECTION 08

Observations and next steps

What the evidence covers and what it does not yet show.

Area	Status	Notes
Register and login	**Covered**	Success paths shown (201, 200)
Category create and list	**Covered**	201 and 200 shown
Validation error (400)	**Covered**	Empty category body
Unknown route (404)	**Covered**	Catch-all handler
Protected route without token (401)	**Not shown**	Send POST /api/categories with no Authorization header
GET /api/auth/me	**Not shown**	Needs Bearer token
Category update and delete	**Not shown**	PUT and DELETE by id, plus delete-while-in-use (400)
Courses (all 5 endpoints)	**Not shown**	Create, list, filter, get, update, delete
Wrong password (401), duplicate email (400)	**Not shown**	Quick extra auth cases

Things worth knowing

- **Ignore the Postman AI panel in Screenshot 4.** It shows a suggestion about a "non-existent route" and "network connection error". That is Postman's own assistant guessing and is unrelated to the result; the API correctly returned 400. If your instructor might be confused, re-take that screenshot with the AI panel closed.
- **Tokens are visible in Screenshots 1a and 1b.** They belong to a throw-away test account, which is fine, but never share real tokens or your real `JWT_SECRET`.
- **Design choice:** an invalid ID format (like `/api/categories/abc`) returns 404 rather than 400. This is consistent and documented in the error handler.
- **Postman requests are saved in one tab named "Post data"** inside "My Collection". Renaming each request (for example "Register", "Login") makes the collection easier to reuse.

Suggested next steps

- Run the missing tests above and add them as extra screenshots, especially courses and the 401 case.
- Save a Postman environment with `baseUrl` and `token` variables so requests are reusable.
- Optional upgrades: pagination on list endpoints, role-based access, rate limiting, and automated tests with Jest and Supertest.

How to run the project

```
npm install
copy .env.example .env      # macOS/Linux: cp .env.example .env
# edit .env: set MONGO_URI and a long random JWT_SECRET
npm run dev                 # starts on http://localhost:5000
```

Make sure MongoDB is running locally, or put a MongoDB Atlas connection string in `MONGO_URI`.